24. FLASH ATTENTION 2

26 March 2026 14:40

Warp = a group of threads executing together on GPU (one parallel worker unit)

Flash Attention is not nearly as efficient as other primitives such as matrix multiply (GEMM)

↓

2x – 4x faster than standard attention

the forward pass only reaches 30 – 50% of theoretical maximum FLOPs/s of the device

backward pass only reaches 25 – 35%

GEMM can reach upto 80 – 90%

Flashattention has suboptimal work partitioning between different thread blocks and warps on the GPU causing either low occupancy or unnecessary shared memory reads/writes

Flash Attention - 2 → better parallelism & work partitioning

to reduce the non-matmul FLOPs (only a fraction of total), they take longer to perform as GPU's have specialized units for matrix multiply

Matmul throughput can be upto 16x higher than non-matmul throughput

So reduce non matmul & spend time on matmul FLOPs

to parallelize both the forward pass and backward pass along the seq length dimension in addition to the batch and no of heads dimension

↳ this increases occupancy (utilization of GPU) in case where sequences are long (and hence batch size is often small)

Even within one block of attention computation, we partition the work between different warps of a thread block to reduce communication and shared memory reads/writes

Attention

$S = QK^T \quad P = \text{softmax}(S) \quad O = PV$

backward pass

dO → gradient O wrt some loss fn

$dV = P^T dO \in \mathbb{R}^{N \times d}$

$dP = dO V^T \in \mathbb{R}^{N \times N}$

$dS = d\,\text{softmax}(dP) \in \mathbb{R}^{N \times N}$

$dQ = dS K \in \mathbb{R}^{N \times d}$

$dK = Q dS^T \in \mathbb{R}^{N \times}$

if $p = \text{softmax}(s)$ for some vector s and p, then with output gradient dp, the input gradient $ds = (\text{diag}(p) - pp^T)dp$

Jacobian of softmax

$s \in \mathbb{R}^n \quad p = \text{softmax}(s)$

Jacobian $\frac{\partial p}{\partial s} = \text{diag}(p) - pp^T$

$\text{diag}(p) = \begin{bmatrix} p_1 & 0 & 0 \\ 0 & p_2 & 0 \\ 0 & 0 & p_3 \end{bmatrix}$ diagonal matrix

Captures self-sensitivity (how p_i changes wrt s_i)

pp^T outer product $\begin{bmatrix} p_1 p_1 & p_1 p_2 \\ p_2 p_1 & p_2 p_2 \end{bmatrix}$

Captures interaction between elements

Math recall

$s = [s_1 \; s_2 \; \dots \; s_n]$

$p_i = \frac{e^{s_i}}{\sum_{k=1}^{n} s_k}$ → input scores

→ output probabilities $\quad \sum p_i = 1$

$\frac{\partial p_i}{\partial s_j}$ how does p_i change if we slightly change s_j

Case 1 $i = j$ $\quad p_i = \frac{e^{s_i}}{\sum_{k=1}^{n} e^{s_k}}$

Use quotient rule & differentiate wrt s_i

$\frac{\partial p_i}{\partial s_i} = \frac{e^{s_i} Z - e^{s_i} e^{s_i}}{Z^2}$ $\quad \left[\frac{vu' - uv'}{v^2}\right]$

$$\begin{bmatrix} P_3 P_1 & P_1 P_2 & P_2 \end{bmatrix}$$

Captures interaction between elements

Softmax is coupled so
increasing s_1 increases P_1
but decreases others

Chain rule
we want $\frac{\partial L}{\partial s}$

But we have $\frac{\partial L}{\partial P} = dP$

$$ds = \left(\frac{\partial P}{\partial s}\right)^T dP$$

Substitute $ds = (\mathrm{diag}(P) - PP^T)\,dP$

Backprop through softmax is not elementwise

each gradient ds_1
depends on dP_1
all other dP_j
again coupling

FA2 written algebraically (matrix form) while FA1 is written procedurally

in FA1 $\ell \to$ normalization
$m_1 \to$ scaling

FA2 writes things like
diag(ℓ)
matrix forms

instead of per-row scalars
they write
diagonal matrices acting on vectors

FAI
Computation mixed
matmul + scalar ops

FAII pushes toward
fewer scalar ops
more batched matrix ops

FA1 → updates normalized output every step
FA2 → keeps unnormalized output, normalizes only at the end

step 1 $$O^{(2)} = \underbrace{\mathrm{diag}\left(\ell^{(1)}/\ell^{(2)}\right)^{-1} O^{(1)}}_{\text{old contribution rescaled}} + \underbrace{\mathrm{diag}\left(\ell^{(2)}\right)^{-1} e^{s^{(2)} - m^{(2)}} V^{(2)}}_{\text{new contribution rescaled}}$$

Step 2
they say we don't need to scale both terms every time

$$\tilde{O}^{(2)} = \mathrm{diag}\left(\ell^{(1)}\right)^{-1} O^{(1)} + e^{s^{(2)} - m^{(2)}} V^{(2)} \Big\} \text{ running numerator}$$

what changed?

FA1 → always divide by ℓ

FA2 → don't divide
→ keep everything in unnormalized space

Step 3
FA1 → every step
multiply
divide
rescale
many non matmul ops

FA2
↳ accumulate
– only matmul + add
normalization delayed

step 4 final normalization

At the end $O = \mathrm{diag}\left(\ell^{(\text{last})}\right)^{-1} \tilde{O}$

$$\frac{\partial P_1}{\partial s_1} = \frac{e^{s_1} z - e^{s_1} e^{s_1}}{z^2} \qquad \left[\frac{vu' - uv'}{v^2}\right]$$

$$= \frac{e^{s_1}}{z}\left[1 - \frac{e^{s_1}}{z}\right]$$

$$\frac{dP_1}{ds_1} = P_1(1 - P_1)$$

Case 2 $i \neq j$ $\quad P_1 = \frac{e^{s_1}}{z}$

wrt s_j

Numerator doesn't depend on s_j

Only denominator changes

$$\text{so } \frac{\partial P_1}{\partial s_j} = \frac{0\,z - e^{s_1} e^{s_j}}{z^2}$$

$$= -\frac{e^{s_1}}{z}\,\frac{e^{s_j}}{z}$$

$$\frac{\partial P_1}{\partial s_j} = -P_1 P_j$$

$$\Rightarrow \frac{\partial P_1}{\partial s_j} = \begin{cases} P_1(1 - P_1) & i = j \\ -P_1 P_j & i \neq j \end{cases}$$

We want a matrix J where $J_{ij} = \frac{\partial P_i}{\partial s_j}$

$$\mathrm{diag}(P) = \begin{bmatrix} P_1 & 0 & 0 \\ 0 & P_2 & 0 \\ 0 & 0 & P_3 \end{bmatrix}$$

$$PP^T = \begin{bmatrix} P_1 P_1 & P_1 P_2 & \cdots \\ P_2 P_1 & P_2 P_2 & \\ \vdots & & \end{bmatrix}$$

$$J = \mathrm{diag}(P) - PP^T$$

Diagonal term $P_1(1 - P_1)$
how P_1 changes with itself

off-diagonal
$-P_1 P_j$
competition
increasing s_1
decreases P_1

Backward pass
upstream gradient $dP = \frac{\partial L}{\partial P}$
$ds = \frac{\partial L}{\partial s}$

Chain rule
$ds = J^T dP$

Since J is symmetric
$ds = (\mathrm{diag}(P) - PP^T)\,dP$

Computationally
$$ds_1 = P_1 dP_1 - P_1 \sum_j (P_j\, dP_j)$$

each ds_1 depends on its own gradient
sum over all elements } coupling

$$\overset{U}{(e^{s_1})}\overset{V}{(1)}$$

$$\frac{e^{s_1} - 0}{(e^{s_1})^2} \Rightarrow \frac{1}{e}$$

why does splitting K, V force synchronization?

At the end $O = \mathrm{diag}(\ell^{(last)})^{-1}\tilde{O}$

Meaning divide once instead of every iteration

Step 5 New stored variable

They introduce $L^{(j)} = m^{(j)} + \log(\ell^{(j)})$

Instead of storing m, ℓ

store log sum exp

Interpretation $\log \sum e^x = m + \log(\ell)$

numerically stable

compact

Algorithm lines

Line 8–9 $S_{ij} = Q_i K_j^T$

Line 9 $m_i^{(j)} = \max(m_i^{(j-1)}, \mathrm{rowmax}(S_{ij}))$

$\ell_i^{(j)} = e^{m^{(j-1)} - m^{(j)}} \ell_i^{(j-1)} + \mathrm{rowsum}(e^{S-m})$

Line 10

$\tilde{O}_i^{(j)} = \mathrm{diag}(e^{m^{(j-1)} - m^{(j)}})\, \tilde{O}_i^{(j-1)} + \tilde{P} V$

No division → only scaling and addition

Step 7 final lines

Normalize once $O_i = \mathrm{diag}(\ell_i^{(T_c)})^{-1} \tilde{O}_i^{(T_c)}$

Compute log sum exp $L_i = m_i + \log(\ell_i)$

FA1 normalize every step, many scalar ops

FA2 accumulate numerator, normalize once, more matmul friendly

Remove normalization from the inner loop (main Algorithmic improvement)

The real speedup comes from {parallelism + work partitioning}

Work partitioning

FA1

They say split K,V across warps

Meaning

each warp

→ computes part of QK^T

→ computes part of PV

After that

all warps must write to shared memory

Synchronize

reduce (sum results)

} problem

Bottleneck

Communication + Synchronization

FA2

They flip it

split Q across warps

Keep K,V shared

$(c^s)^r$ 2

own gradient

sum over all elements } coupling

{ Why does splitting K,V force synchronization?

{ Why does splitting Q eliminate it?

Step 1 what are we computing?

$O = \mathrm{softmax}(QK^T)V$

focus on one row $O_i = \sum_j \mathrm{softmax}(S_{ij}) V_j$

Step 2 FA1 (split K,V across warps)

warp 1 → uses part of K,V

warp 2 → uses another part

Each warp computes partial output

warp 1 → contribution from V_1

warp 2 → contribution from V_2

final output needs O_i = sum of ALL contributions

So, All warps must

1. Write partial results
2. Synchronize
3. Sum them

Communication + Synchronization is the cost

FA2 (split Q across warps)

Now warp 1 → handles Q_1

warp 2 → handles Q_2

Each warp computes

O_i = full output for its own row

Key diff each warp uses all K,V produces complete result

So no need to combine with others

Synchronize

FA1 many workers → each does partial → must combine

FA2 each worker → does full job → no combine

Most significant/dominant reason for FA2 >> FA1

Between → fewer non-matmul ops

better parallelism

less synchronization

Most dominant is

eliminating inter warp synchronization

(better parallelization/utilization)

GPU are massively parallel

1 threads wait,

sync

communicate

GPU stalls → throughput drops

FA2 fixes this by

→ giving each warp independent work

→ no waiting

→ no reduction

full hardware utilization

Contrast

FA1	FA2
split KV	split Q
partial outputs	full outputs
need reduction	no reduction
sync overhead	no sync

This removes shared memory traffic

synchronization stalls

Way forward

Split Q across warps

Keep K,V shared

Meaning each warp

owns a slice of Q

computes full result for that slice

Result no need to

communicate between warps

reduce intermediate outputs

Each warp produces its own final output independently

Synchronisation stalls